

EYP1113-L Probabilidades y Estadísticas

Control 01

Ayuda en R

```
?palabra #help  
rm(mi_objeto) #borra un objeto de entorno
```

Operadores lógicos

```
< #menor  
< #mayor  
>= o <= #menor/mayor o igual  
== #igual  
!= #diferente  
$ #and  
| #or
```

Operadores matemáticos

```
a+b #suma  
a-b #resta  
a^b #exponente  
a*b #multiplicación  
a%%b #resto de división  
sqrt(x) #raíz cuadrada de x  
logn(x) #logaritmo base n de x  
log(x) #logaritmo de base natural  
exp(x) #función exponencial de x  
abs(x) #valor absoluto de x  
cos(x) #coseno de x  
sen(x) #seno de x  
choose(a,b) #coeficiente binominal  
factorial(n) #factorial de n
```

Funciones de R

```

#variables
variable = "hello" #declaración de variables
variable_booleana_verdadera = TRUE
variable_booleana_verdadera = FALSE

#vectores
c() #vector de objetos
v[i] #para acceder al i-ésimo componente del vector:
suma() #suma entre todos los componentes del vector
prod() #producto entre todos los componentes del vector
mean() #promedio entre todos los componentes del vector
sd() #desviación estándar entre todos los componentes del vector
min() #mínimo valor del vector
max() #máximo valor del vector
length() #largo vector

#matrices
matrix() # Para definir una matriz
nrow = # Cantidad de filas
ncol = # Cantidad de columnas
#ejemplo: matrix(v3, nrow=4, ncol=5, byrow=TRUE)
m[i,j] # Para acceder a un elemento de la matriz en la posición [i,j]
m[i,] # Para acceder a la fila i de la matriz
m[,j] # Para acceder a la columna j de la matriz

#tablas
table(bd$Artist)
table(bd$Artist, bd$Artist)
table[1,3] #para acceder a dato en posición 1,3

#listas
ls() #lista de objetos
lista[i] o lista$objeto #para acceder a los objetos de la lista

#funciones
while(condición){}
for(condición){}
if(condición){}
else(){}

```

Importación de librerías

```

install.packages("ggplot2") #instalación de librería
library(rio) #inicio librería rio

```

```
library(dplyr) #inicio libreria dplyr
```

Importación de base de datos

```
#importación en PositCloud
ruta = file.choose() #generamos ruta de base de datos
nombre_base_datos = import(ruta) #importamos datos

#importación en RStudio
install.packages("rio")
library(rio)
ruta = file.choose() #generamos ruta de base de datos
nombre_base_datos = import(ruta) #importamos datos

#alternativa es usar base datos en linea, o sea:
data(mtcars)
```

Vista previa base de datos

```
head(bd) # Muestra las primeras filas de bd
tail(bd) # Muestra las últimas filas de bd
str(bd) # Muestra la estructura de bd
names(bd) # Lista los nombres de las columnas
```

Manipulación de base de datos

```
#convertir base de dato a formato xlsx:
install.packages("writexl")
library(writexl)
fish_data <- read.csv("fish.csv") #leer el archivo CSV
write_xlsx(fish_data, path = "fish.xlsx") #guardar el archivo en formato Excel

#genera un dataframe de la base de dato
data.frame(variable1, variable2,...) #crea una base de dato
data.frame(Nombre1=variable1,...) #define nombre de columnas

#genera objeto correspondiente a linea/columna n de base de dato:
filtro_linea1 <- bd[n,]
filtro_columna1 <- bd[,n]

# elimina linea/columna n de base de dato:
borrar_primera_fila <- bd[-1, ]
borrar_primera_columna <- bd[, -1]
```

```

#selección de datos dado una condición:
filtrar_maroon5 <- bd[bd$Artist == 'Maroon 5', ] #genera objeto con datos de
columna artista de base de dato que cumplan condición
cut(bd$Artist, breaks = c(10,20,300, labels = c("Bajo","Media","Alto")))
#filtra datos cualitativos dado una condición

# Convertir datos:
factor_cyl <- as.factor(mtcars$cyl) # Convierte cyl en variable cualitativa
numeric_cyl <- as.numeric(mtcars$cyl) # Convierte cyl en variable cuantitativa

#renombrar valores
rownames() #para renombrar filas
colnames() #para renombrar columna:
rownames()[i] #para renombrar una fila en específico
colnames()[j] #para renombrar una columna en específico

nrow() #Devuelve el número de filas.
ncol() #Devuelve el número de columnas.
dim() #Devuelve un vector con el número de filas y columnas.

```

Medidas descriptivas y estadísticas de base de datos

```

#resumen estadístico
resumen_estadistico_mpg <- summary(mtcars$mpg)

#medidas de valor
minimo_mpg <- min(mtcars$mpg) #mínimo
maximo_mpg <- max(mtcars$mpg) #máximo

#medidas de valores centrales
media_mpg <- mean(mtcars$mpg) #media
moda_mpg <- modeest::mlv(mtcars$mpg) #moda
mediana_mpg <- median(mtcars$mpg) #mediana

#medidas de posición
cuantiles_mpg <- quantile(mtcars$mpg, probs = c(0.25, 0.5, 0.75)) #cuartiles

#medidas de dispersión
varianza_mpg <- var(mtcars$mpg) #varianza
desviacion_mpg <- sd(mtcars$mpg) #desviación
covarianza_mpg_cyl <- cov(mtcars$mpg, mtcars$cyl) #covarianza entre mpg y cyl
correlacion_mpg_cyl <- cor(mtcars$mpg, mtcars$cyl) #correlación entre mpg y
cyl

```

```
#medidas de asimetría
asimetria_mpg <- moments::skewness(mtcars$mpg) #Skewness
curtosis_mpg <- moments::kurtosis(mtcars$mpg) #Kurtosis
```

Medidas gráficas de base de datos

```
hist(casillas, breaks = 5, freq = F, xlim = c(0,1), ylim = c(0,3)) #histograma
beta = curve(dbeta(x, shape1 = 10, shape2 = 10), from = 0, to = 1, add = TRUE,
col = "red", lwd = 2) #grafica distribución beta

#características gráficas
xlab="Nombre de eje" # Para agregar nombre en el eje X
ylab="Nombre de eje" # Para agregar nombre en el eje Y
main="Título" # Para agregar título al gráfico
col="Nombre del color" # Para cambiar el color del gráfico
lwd= # Para cambiar el grosor de las líneas
text(x,y,label="texto") # Para agregar texto en un punto (x, y)
points(x1,y1,...) # Para agregar un punto en (x1, y1) después de ejecutar
plot()
abline(h= o v=) # Para agregar líneas horizontales (h=) y/o verticales (v=)
organizados en 1 fila y n columnas
```

Modelos de probabilidad

```
binom() #distribución Binominal
beta()
pois() #distribución Poisson
unif() #distribución Uniforme
norm() #distribución Normal
exp() #distribución Exponencial
gamma() #distribución Gamma
chisq() #distribución Chi-Cuadrado
t() #distribución t-Student
f() #distribución Fisher
```

Cálculo de probabilidad de modelos de una variable

```
ddist(x,...) #entrega probabilidad en x
pdist(q,...) #entrega probabilidad acumulada hasta q
qdist(p,...) #entrega valor de x talque tenga dicha probabilidad
rdist(n,...) #genera una muestra aleatoria de dicha distribución
```

Control 2

Modelos de probabilidad conjunta

```
# Crear tabla de datos modo 1:
X <- c(-1, 0, 1)
Y <- c(0, 1, 2, 3)

prob <- c(0.04, 0.16, 0.04, 0.12,
          0.16, 0.12, 0, 0,
          0.12, 0.16, 0, 0.08)

tabla <- matrix(prob, nrow = 3, byrow = TRUE)

rownames(tabla) <- paste("X =", X)
colnames(tabla) <- paste("Y =", Y)

tabla

# Crear tabla de datos modo 2:
tabla <- matrix(c(0.04, 0.16, 0.04, 0.12,
                  0.16, 0.12, 0, 0,
                  0.12, 0.16, 0, 0.08),
               nrow = 3, byrow = TRUE)

rownames(tabla) <- c("X = -1", "X = 0", "X = 1")
colnames(tabla) <- c("Y = 0", "Y = 1", "Y = 2", "Y = 3")

tabla

# Crear tabla de datos modo 3:
tabla_conjunta <- table(heightweight$sex, heightweight$height_category)
tabla_conjunta

# Ojo en este ultimo caso, se debe convertir datos a probabilidades conjuntas:
prob_conjunta <- prop.table(tabla_conjunta)
round(prob_conjunta, 3)

# Obtener probabilidades:
sum(tabla["X = -1",]) # P (X = -1)
tabla["X = 0", "Y = 2"] # P (X = 0 | Y = 2)

# P (Y < 2 | X = 1):
prob_conjunta <- sum(tabla["X = 1", c("Y = 0", "Y = 1")])
prob_marginal_X1 <- sum(tabla["X = 1", ])
prob_condicional <- prob_conjunta / prob_marginal_X1
```

prob_condicional

```
# Probabilidades de modelos:
# 1. Probabilidad Conjunta  $P(X = x, Y = y)$ 
# Para calcular la probabilidad conjunta de dos variables X y Y, usamos:
#  $P(X = x, Y = y) = P(X = x) * P(Y = y | X = x)$ 

# Ejemplo:  $X \sim \text{Poisson}(\text{lambda})$ ,  $Y | X = x \sim \text{Binomial}(x, p)$ 
#  $P(X = x, Y = y) = P(X = x) * P(Y = y | X = x)$ 

# Definir los parámetros
lambda <- 7 # Parámetro Poisson
x <- 7      # Valor de X
y <- 4      # Valor de Y
p <- 0.3    # Probabilidad de éxito en la distribución Binomial

#  $P(X = 7)$  para Poisson
P_X <- dpois(x, lambda)

#  $P(Y = 4 | X = 7)$  para Binomial
P_Y_given_X <- dbinom(y, size = x, prob = p)

# Probabilidad conjunta
P_XY <- P_X * P_Y_given_X
P_XY

# 2. Probabilidad Condicional  $P(X = x | Y = y)$ 
# Usamos la fórmula de Bayes para la probabilidad condicional:
#  $P(X = x | Y = y) = P(X = x, Y = y) / P(Y = y)$ 

# Cálculo de  $P(X = 10 | Y = 7)$  como ejemplo

#  $P(X = 10, Y = 7)$ 
P_X_10 <- dpois(10, lambda)
P_Y_7_given_X_10 <- dbinom(7, size = 10, prob = p)
P_XY_10_7 <- P_X_10 * P_Y_7_given_X_10

#  $P(Y = 7)$  sumando las probabilidades conjuntas para todos los valores de X
P_Y_7 <- sum(sapply(0:20, function(k) dpois(k, lambda) * dbinom(7, size = k,
prob = p))))

#  $P(X = 10 | Y = 7)$ 
P_X_given_Y <- P_XY_10_7 / P_Y_7
P_X_given_Y

# 3. Probabilidad Condicional Acumulada  $P(Y \leq y | X = x)$ 
```

```

# Para calcular la probabilidad condicional acumulada, usamos la distribución
acumulada:
#  $P(Y \leq y \mid X = x) = \sum(P(Y = k \mid X = x))$  para  $k=0$  hasta  $y$ 

# Ejemplo:  $P(Y \leq 4 \mid X = 9)$ 
P_Y_leq_4_given_X <- pbinom(4, size = 9, prob = 0.3)
P_Y_leq_4_given_X

# 4. Probabilidad de Eventos Condicionales
# Ejemplo con  $X \sim \text{Poisson}(7)$  y  $Y \mid X = x \sim \text{Binomial}(x, p)$ 

#  $P(X = x, Y = y)$  para Poisson y Binomial
P_XY <- dpois(x, lambda) * dbinom(y, size = x, prob = p)

#  $P(Y \leq 4 \mid X = 9)$  usando binomial acumulada
P_Y_leq_4_given_X <- pbinom(4, size = 9, prob = 0.3)

#  $P(X = 10 \mid Y = 7)$  usando Bayes
P_X_given_Y <- sum(sapply(0:10, function(x_val) dpois(x_val, lambda) *
dbinom(7, size = x_val, prob = 0.3))) / sum(dpois(0:20, lambda) * dbinom(7,
size = 0:20, prob = 0.3))
P_X_given_Y

# 5. Problemas de probabilidad con diferentes distribuciones

# Ejemplo de Poisson y Binomial
#  $X \sim \text{Poisson}(\lambda)$  y  $Y \mid X = x \sim \text{Binomial}(x, p)$ 
# Para calcular probabilidades conjuntas y condicionales

# Cálculo de probabilidades en R
#  $P(X = 7, Y = 4)$ 
P_XY <- dpois(7, lambda) * dbinom(4, size = 7, prob = 0.3)

#  $P(Y \leq 4 \mid X = 9)$ 
P_Y_leq_4_given_X <- pbinom(4, size = 9, prob = 0.3)

#  $P(X = 10 \mid Y = 7)$ 
P_X_given_Y <- sum(sapply(0:10, function(x_val) dpois(x_val, lambda) *
dbinom(7, size = x_val, prob = 0.3))) / sum(dpois(0:20, lambda) * dbinom(7,
size = 0:20, prob = 0.3))

# 6. Códigos útiles en R para estos cálculos

# Distribución Poisson
dpois(x, lambda) # Probabilidad de  $X = x$ 
ppois(q, lambda) # Probabilidad acumulada  $P(X \leq q)$ 

```

```
# Distribución Binomial
dbinom(x, size, prob) # Probabilidad de  $Y = x$ 
pbinom(q, size, prob) # Probabilidad acumulada  $P(Y \leq q)$ 

# Estimación de máxima verosimilitud (EMV)
optim(par = valor_inicial, fn = func_log_verosimilitud)
```

Simulación de variables aleatoria

```
set.seed(1010) # Fijar semilla para reproducibilidad

alpha <- 6 # Parámetro  $\alpha$  de la Beta
beta <- 2 # Parámetro  $\beta$  de la Beta
lambda <- 1 # Parámetro  $\lambda$  de las Gammas (valor por defecto, ya que no afecta a la Beta)

n <- 300
X <- rgamma(n, shape = alpha, rate = lambda)
Y <- rgamma(n, shape = beta, rate = lambda)

Beta_samples <- X / (X + Y)

hist(Beta_samples, freq = FALSE, col = "lightblue",
     main = "Histograma de Beta(6, 2)",
     xlab = "Valores Beta(6, 2)", ylab = "Densidad")

curve(dbeta(x, shape1 = alpha, shape2 = beta),
     col = "red", lwd = 2, add = TRUE)
```

Simulación de varias simulaciones de función de variables aleatoria

```
set.seed(24) # Fijar semilla para reproducibilidad

# Parámetros de la distribución Gamma
n <- 10 # Tamaño de cada muestra
k <- 200 # Número de simulaciones
shape <- 3 # Parámetro  $\alpha$  de la Gamma
rate <- 3 # Parámetro  $\lambda$  de la Gamma

# Simulación de k muestras aleatorias de tamaño n
X <- matrix(rgamma(n * k, shape = shape, rate = rate), nrow = k, ncol = n)
```

```
# Aplicación de la transformación  $Y = (1/10) * \sum(\log(X_i))$ 
Y <- (1 / 10) * rowSums(log(X))

# Visualización de la distribución de Y
hist(Y, freq = FALSE, col = "lightblue", main = "Histograma de Y",
      xlab = expression(Y == frac(1, 10) * sum(log(X[i]))), ylab = "Densidad")
lines(density(Y), col = "red", lwd = 2, add = True) # Agregar densidad
estimada
```

Estimación máximo: función de log-verosimilitud:

```
set.seed(2024)

# Parámetros de la distribución Gamma
alpha <- 2
beta <- 2
n <- 500

# Generar muestra de la distribución Gamma
gamma <- rgamma(n, shape = alpha, rate = beta)

# Definir la función de log-verosimilitud
l.gamma <- function(beta) {
  -sum(dgamma(gamma, shape = alpha, rate = beta, log = TRUE)) # Sumar log-
verosimilitudes
}

# Crear un rango para graficar beta
grid <- seq(0.1, 4, length = 1000)

# Evaluar la función de log-verosimilitud y graficar
plot(grid, sapply(grid, l.gamma), type = "l", lwd = 2,
      main = "Log-Verosimilitud Gamma", xlab = "Beta", ylab = "-Log-
Verosimilitud")

# Calcular el EMV para beta
EMV.gamma <- optim(par = 3, fn = l.gamma, method = "L-BFGS-B", lower = 0.1,
upper = 10)$par

# Comparar el EMV con el valor real de beta
diferencia <- EMV.gamma - beta
diferencia

#### Estimación máximo: función de log-verosimilitud:
```r
set.seed(2024)
```

```

Parámetros de la distribución Gamma
alpha <- 2
beta <- 2
n <- 500

Generar muestra de la distribución Gamma
gamma <- rgamma(n, shape = alpha, rate = beta)

Definir la función de log-verosimilitud
l.gamma <- function(beta) {
 -sum(dgamma(gamma, shape = alpha, rate = beta, log = TRUE)) # Sumar log-
verosimilitudes
}

Crear un rango para graficar beta
grid <- seq(0.1, 4, length = 1000)

Evaluar la función de log-verosimilitud y graficar
plot(grid, sapply(grid, l.gamma), type = "l", lwd = 2,
 main = "Log-Verosimilitud Gamma", xlab = "Beta", ylab = "-Log-
Verosimilitud")

Calcular el EMV para beta
EMV.gamma <- optim(par = 3, fn = l.gamma, method = "L-BFGS-B", lower = 0.1,
upper = 10)$par

Comparar el EMV con el valor real de beta
diferencia <- EMV.gamma - beta
diferencia

```

## Estimación por fitdist

```

library(fitdistrplus)
x = Abalon$largo #Base de datos
mean(x);sd(x)

estimacion = fitdist(x,distr = "gamma", method = "mle")
estimacion$estimate

Ojo, en method tenemos: method = "mme" para Method of Moments Estimation y
method = "mle" Maximum Likelihood Estimation.
Ojo, en distr tenemos: Distributions `"beta"`, `"cauchy"`, `"chi-squared"`,
`"exponential"`, `"gamma"`, `"geometric"`, `"log-normal"`, `"lognormal"`,
`"logistic"`, `"negative binomial"`, `"normal"`, `"pois"`, `"t"` and

```

```
`"weibull"` are recognised, case being ignored.
Ojo, algunas distribuciones como las
```

## Intervalo de confianza

```
Datos ejemplo
diam <- c(3.236, 3.223, 3.242, 3.244, 3.228, 3.253, 3.253, 3.230)
n <- length(diam)
alpha <- 0.05

Intervalo usando t-Student
liminf1 <- mean(diam) + qt(alpha / 2, df = n - 1) * sqrt(var(diam) / n)
limsup1 <- mean(diam) + qt(1 - alpha / 2, df = n - 1) * sqrt(var(diam) / n)

Intervalo usando Normal
liminf2 <- mean(diam) - qnorm(1 - alpha / 2) * sqrt(var(diam) / n)
limsup2 <- mean(diam) + qnorm(1 - alpha / 2) * sqrt(var(diam) / n)

Resultados
c(liminf1, limsup1) # Intervalo t-Student
c(liminf2, limsup2) # Intervalo Normal
```

## Distribución Normal bivariada

```
Cálculos de probabilidades usando mvtnorm
library(mvtnorm)

Parámetros de la normal bivariada
mu <- c(2, 2) # Medias de X e Y
sigma <- matrix(c(1^2, 0.3 * 1 * 2, # Varianza de X y covarianza
 0.3 * 1 * 2, 2^2), # Covarianza y varianza de Y
 nrow = 2)

(a) $P(1 < X < 2; 1 < Y < 3)$
lower_a <- c(1, 1) # Límites inferiores
upper_a <- c(2, 3) # Límites superiores
prob_a <- pmvnorm(lower = lower_a, upper = upper_a, mean = mu, sigma = sigma)
print(prob_a)

(b) $P(X < 2.3)$
mu_X <- 2
sigma_X <- 1
prob_b <- pnorm(2.3, mean = mu_X, sd = sigma_X)
print(prob_b)
```

```

(c) $P(X > 1; Y < 2.3)$
lower_c <- c(1, -Inf) # Límites inferiores
upper_c <- c(Inf, 2.3) # Límites superiores
prob_c <- pmvnorm(lower = lower_c, upper = upper_c, mean = mu, sigma = sigma)
print(prob_c)

#####
Visualización de la Normal Bivariada
#####

Parámetros para la normal bivariada estándar
x.axis <- seq(-3, 3, length = 50)
y.axis <- seq(-3, 3, length = 50)

Función para calcular densidad bivariada
d2norm <- function(x, y){
 mu <- c(0, 0) # Medias
 cov <- matrix(c(1, 0, 0, 1), ncol = 2) # Matriz de covarianza
 z <- numeric(length(x)) # Inicializar vector para las densidades
 for (i in 1:length(x)){
 z[i] <- dmvnorm(c(x[i], y[i]), mean = mu, sigma = cov) # Evaluar densidad
 }
 return(z)
}

Calcular densidad bivariada para una cuadrícula de valores
z.axis <- outer(x.axis, y.axis, FUN = d2norm)

Gráficos
Superficie 3D
persp(x.axis, y.axis, z.axis, theta = 100, phi = 20, expand = 1, shade = 0.1,
col = "orange",
 main = "Superficie de Densidad Normal Bivariada")

Imagen 2D
image(x.axis, y.axis, z.axis, main = "Densidad Normal Bivariada (Imagen 2D)",
xlab = "X", ylab = "Y")

Contornos
contour(x.axis, y.axis, z.axis, add = TRUE, col = "darkred", lwd = 1, main =
"Contornos de la Densidad")

```

# Teorema del Límite Central (TLC): simulación con 200 variables i.i.d. y aplicación del TLC

```
Establecer la semilla para reproducibilidad
set.seed(123)

Parámetros
n <- 1000 # Número de muestras
size <- 30 # Tamaño de cada muestra

Generar una población de datos uniformes entre 0 y 1
poblacion <- runif(n * size, min = 0, max = 1)

Dividir la población en muestras de tamaño 'size'
Xn <- matrix(poblacion, nrow = n, ncol = size)

Calcular la suma de cada muestra
Sn <- apply(Xn, MARGIN = 1, FUN = sum)

Graficar la distribución de las sumas muestrales
hist(Sn, breaks = 30, main = "Distribución de las sumas muestrales",
 xlab = "Suma de la muestra", col = "skyblue", border = "black")

Graficar la distribución de la población original
hist(poblacion, breaks = 30, main = "Distribución de la población original",
 xlab = "Valor", col = "orange", border = "black")

Añadir líneas de la media y la desviación estándar
abline(v = mean(Sn), col = "red", lwd = 2) # Media de las sumas muestrales
abline(v = mean(poblacion), col = "green", lwd = 2) # Media de la población

Mostrar resumen estadístico de las sumas
summary(Sn)

Mostrar resumen estadístico de la población
summary(poblacion)
```

## TCL:

```
Cargar los datos
data <- read.csv("ruta_a_tu_base_de_datos.csv") # Si los datos están en un
archivo CSV
O si ya tienes la base cargada en R:
data <- tu_base_de_datos
```

```

Seleccionar la columna de interés
x <- data$columna_interes # Reemplaza 'columna_interes' por el nombre real de
la columna

Asegurar resultados reproducibles y definir parámetros
set.seed(123) # Reproducibilidad
n <- 30 # Tamaño de cada muestra aleatoria
num_samples <- 1000 # Número de muestras que se tomarán

Verificar la distribución original de la columna
hist(x, breaks = 30, main = "Distribución Original", xlab = "Valores", col =
"skyblue")

Aplicar el TCL: tomar medias de múltiples muestras
sample_means <- replicate(num_samples, mean(sample(x, n, replace = TRUE)))

Graficar los resultados
hist(sample_means, breaks = 30, main = "Aproximación Normal (TCL)",
xlab = "Medias muestrales", col = "lightgreen", probability = TRUE)

Comparar con la distribución normal teórica
mean_x <- mean(x, na.rm = TRUE) # Calcular la media ignorando NA's si hay
sd_x <- sd(x, na.rm = TRUE) / sqrt(n) # Desviación estándar muestral
curve(dnorm(x, mean = mean_x, sd = sd_x), col = "red", lwd = 2, add = TRUE)

Agregar leyenda
legend("topright", legend = c("Teórico Normal"), col = c("red"), lwd = 2)

```

## Test de Hipótesis:

```

Cargar las librerías necesarias
Si no tienes fitdistrplus instalado, puedes hacerlo con:
install.packages("fitdistrplus")
library(fitdistrplus)

Datos de ejemplo (reemplaza Abalon$largo y Abalon$anillos con tus datos
reales)
x1 = Abalon$largo # Suponiendo que Abalon$largo contiene los datos del largo
de los locos
x2 = Abalon$anillos # Suponiendo que Abalon$anillos contiene los datos del
número de anillos

Nivel de significancia

```

```

alpha = 0.05

1. **Prueba t de una muestra**: Comparar la media de una muestra con un
valor específico (14 cm)
t_test_result_1 <- t.test(x1, mu = 14, alternative = "less", conf.level = 1 -
alpha)
print("Resultado de la prueba t de una muestra:")
print(t_test_result_1)

2. **Prueba t de dos muestras**: Comparar las medias de dos muestras
independientes
Generando dos muestras aleatorias de distribuciones Gamma como ejemplo
n1 = 100 # Tamaño de la muestra 1
n2 = 80 # Tamaño de la muestra 2
x1_sample = rgamma(n1, 2, 2) # Muestra 1 (distribución Gamma)
x2_sample = rgamma(n2, 4, 3) # Muestra 2 (distribución Gamma)

t_test_result_2 <- t.test(x1_sample, x2_sample, alternative = "two.sided",
var.equal = FALSE)
print("Resultado de la prueba t de dos muestras:")
print(t_test_result_2)

3. **Prueba F para varianzas**: Comparar las varianzas de dos muestras
var_test_result <- var.test(x1_sample, x2_sample, ratio = 1, alternative =
"two.sided", conf.level = 1 - alpha)
print("Resultado de la prueba F para varianzas:")
print(var_test_result)

4. **Ajuste de distribución Poisson utilizando el método de máxima
verosimilitud (MLE)** para número de anillos
Ajustar la distribución Poisson a los datos de "anillos"
poisson_fit <- fitdist(x2, "poisson", method = "mle")
print("Ajuste de distribución Poisson (MLE):")
print(poisson_fit$estimate)

5. **Ajuste de distribución Gamma utilizando el método de máxima
verosimilitud (MLE)** para largo de locos
Ajustar la distribución Gamma a los datos de "largo"
gamma_fit <- fitdist(x1, "gamma", method = "mle")
print("Ajuste de distribución Gamma (MLE):")
print(gamma_fit$estimate)

```

## Modelo de regresión lineal simple:

```

Supongamos que tienes un data.frame 'bd' con las columnas
'Superficie_Construida_M2' y 'Valor_UF'

1. Diagrama de dispersión entre superficie construida y valor UF
plot(bd$Superficie_Construida_M2, bd$Valor_UF,
 xlab = 'Superficie construida', ylab = 'Valor UF',
 cex = 1.5, pch = 20)

2. Calcular la correlación entre las variables
correlation <- cor(bd$Superficie_Construida_M2, bd$Valor_UF)
print(paste("Correlación entre superficie construida y valor UF:",
correlation))

3. Ajuste del modelo de regresión lineal
mod <- lm(Valor_UF ~ Superficie_Construida_M2, data = bd)
Ojo, Valor UF es la variable dependiente, y Superficie construida es la
variable independiente a la que buscaremos su relación.

Ver el modelo ajustado
print(mod)

Agregar la recta ajustada al diagrama de dispersión
plot(bd$Superficie_Construida_M2, bd$Valor_UF,
 xlab = 'Superficie construida', ylab = 'Valor UF',
 cex = 1.5, pch = 20)
abline(mod, col = 'red')

4. Resumen del modelo para ver la significancia
summary_mod <- summary(mod)
print(summary_mod)

5. Determinar el valor crítico para $\alpha = 0.05$
alpha <- 0.05
n <- nrow(bd) # Número de observaciones en el dataset
t_critical <- qt(1 - alpha / 2, df = n - 2)
print(paste("Valor crítico para $\alpha = 0.05$:", t_critical))

6. Veredicto: Si el p-valor del predictor es menor que 0.05, el predictor es
significativo
p_value <- summary_mod$coefficients[2, 4]
if (p_value < alpha) {
 print("El predictor es significativo.")
} else {
 print("El predictor no es significativo.")
}

```

## Modelo de regresión lineal multiple:

```
Cargar un conjunto de datos de ejemplo
data(mtcars)

1. Visualizar las primeras filas de la base de datos
head(mtcars)

2. Definir el modelo lineal múltiple
Queremos predecir el consumo (mpg) usando el peso (wt) y la potencia (hp)
modelo <- lm(mpg ~ wt + hp, data = mtcars)

3. Resumen del modelo
summary(modelo)
```

## Supuesto 2: Homocedasticidad

```
Verificación de homocedasticidad con residuos studentizados
par(mfrow = c(2, 2))
plot(bd$Superficie_Construida_M2, t, main = 'Res. studentizados vs Predictor',
 pch = 19)
abline(h = 0, col = 'red', lty = 2)
plot(y_gorro, t, main = 'Res. studentizados vs Ajustados', pch = 19)
abline(h = 0, col = 'red', lty = 2)
dev.off()
```

## Supuesto 3: Normalidad

```
Verificación de normalidad con histograma y Q-Q plot
par(mfrow = c(1, 2))
hist(r, main = "Histograma de residuos estandarizados", freq = FALSE)
qqnorm(r, main = "Q-Q Plot de residuos")
qqline(r, col = 'red', lty = 1)

Prueba de normalidad de Lilliefors
lillie.test(e)
```

## Supuesto 4 y 5: Independencia e igualdad de distribución

```
Verificación de independencia con residuos
par(mfrow = c(1, 2))
```

```

plot(r, main = 'Residuos estandarizados', ylim = c(-2.5, 2.5), pch = 19)
abline(h = 0, col = 'red', lty = 2)
plot(t, main = 'Residuos studentizados', ylim = c(-2.5, 2.5), pch = 19)
abline(h = 0, col = 'red', lty = 2)
dev.off()

Test de Durbin-Watson para independencia
lmtest::dwtest(mod)

```

## Intervalo de predicción

```

Predicción para una casa de 200 m²
x0 <- 200
predict(mod, data.frame(Superficie_Construida_M2 = x0), interval =
"prediction")

Predicción con intervalos de confianza
pred <- predict(mod, interval = 'prediction')
plot(bd$Superficie_Construida_M2, bd$Valor_UF, pch = 20, cex = 1.5)
abline(mod, col = 'red')
lines(sort(bd$Superficie_Construida_M2), pred[, 2]
[order(bd$Superficie_Construida_M2)], lty = 2, col = 'red')
lines(sort(bd$Superficie_Construida_M2), pred[, 3]
[order(bd$Superficie_Construida_M2)], lty = 2, col = 'red')

```